Knowledge Distillation and Deployment of a Lightweight Command-Parsing Model on a Local Cloud

Cloud Computing (CS3081) mini project · **Arunendra Tripathi**, Roll No. **230103016**, IIIT Manipur · Repository: https://github.com/Arunendra21/kd-command-parser (commit tagged `final`) · Model releases: https://github.com/Arunendra21/kd-command-parser/releases

Labels: Measured = a number read from a file in `results/` (the builder `report/build_report.py` fills every one of them); Assumption = a stated assumption, not a measurement.

0. System overview

The teacher **phi3.5** (Phi-3.5-mini-instruct, 3.8B, Ollama, temperature 0, seed 42) labels 3,000 SLURP commands with a four-key JSON frame. The student **EleutherAI/pythia-70m** is fine-tuned on the teacher's replies (sequence-level KD) and served by a FastAPI service. Everything runs on one single-node **k3d** cluster (k3s v1.30.6) on the laptop used for this project (8 vCPU, 15 GiB RAM, Docker), in three namespaces: `data` (SeaweedFS S3 with buckets `datasets` and `models`), `pipeline` (teacher workers as an Indexed Job) and `app` (Deployment, Service, Traefik Ingress, HPA). The laptop could not run the 3.8B teacher over 3,000 commands in the available time, so Ollama ran on a **free Google Colab T4** and was reached by the in-cluster workers through a free Cloudflare quick tunnel (`OLLAMA_URL`); model, prompt and settings are unchanged. Training and evaluation ran on the laptop CPU (4 cores). CI runs on free GitHub Actions. Because another program occupies host port 8080 on that laptop, the local cluster mapped the Ingress to host port 28080; `cluster/k3d.yaml` in the repository uses 8080, so the standard Locust command works unchanged on a fresh machine.

Task 1: Object storage and parallel teacher labelling

Architecture. `make up` creates the cluster from `cluster/k3d.yaml`, deploys SeaweedFS (single `weed server -s3` pod, PVC, NodePort) in `data`, creates both buckets and uploads `commands_pool.jsonl` to `datasets` (`results/storage_init.log`). S3 credentials are generated per cluster into the git-ignored `.secrets/` and loaded as Kubernetes Secrets. The prompt is `prompts/teacher_prompt.txt` (eight intents with meanings, the three slot rules, six examples taken from the handout, "JSON only"); it is mounted into the workers from a ConfigMap, so the workers always use the committed prompt. Ollama is called with `format: "json"`.

Sharding. `worker/worker.py` reads `SHARD_INDEX` and `SHARD_COUNT` and keeps the commands whose 0-based line number n satisfies $n \bmod \texttt{SHARD_COUNT} = \texttt{SHARD_INDEX}$. The workers run as one **Indexed Job** in `pipeline` with `completions = parallelism = 3`; each pod's `SHARD_INDEX` is its completion index (downward API), so the three shards are disjoint and together cover every line. Modulo (rather than contiguous blocks) balances the load even if the file is sorted, and needs no knowledge of the file length. Each worker writes its own object `datasets/teacher/shard-<i>.jsonl` and checkpoints every 25 records, so a restarted pod resumes instead of starting over (useful because the teacher sits behind a tunnel). `scripts/check_shards.py` confirmed Measured 3 × 1,000 records, 3,000 unique ids, 0 missing, 0 repeated. Wall time Measured: about 26 min for 3,000 commands (3 workers, ~1.5 s per request each).

Cleaning. `worker/clean.py` reads all shards, sorts by id and applies rules (a)-(d) in order (a record is counted under the first rule it breaks), then removes duplicate commands among the survivors (text lower-cased and whitespace-normalised; smallest id kept). Slot rule (d) accepts null or a non-empty string that occurs in the command (case/whitespace-insensitive). Each kept line stores the teacher's reply unchanged (`teacher_raw`) plus the same frame re-serialised on one line (`target`), which is the training target. Re-running `clean.py` on the committed shard files reproduces the checksum in `train.sha256`: `d80d7e7bc4073d80…` Measured.

Input	(a) not JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	Duplicates	Kept	Check
3000	0	1	3	139	46	2811	0+1+3+139+46+2811 = 3000 Measured

Limitations / improvements. (d) is the main loss (139 records): the teacher sometimes normalises values (e.g. "7 am" for "seven am") despite the prompt. A teacher on a remote GPU through a tunnel is a single point of failure (mitigated by checkpoints and retries). Improvement: run Ollama in the cluster on a GPU node, and add a few-shot block with more normalisation counter-examples.

Task 2: Training the student (sequence-level distillation)

Implementation. `training/train.py` (PyTorch + Transformers, CPU) reads only `training/config.yaml` (seed 42, learning rate 3e-4, batch 16, 10 epochs, max length 128, library versions) and trains on the first 1,500 records of `train.jsonl` (its SHA-256 is verified first). Each example is `Command: <text>\nJSON: <teacher JSON><eos>`; the loss is computed only on the JSON part (prompt tokens masked). The seed is fixed in code (Python, NumPy, PyTorch, data-loader generator). `make train SUBSET=50` trains on 50 records. v2 = the same settings with one more epoch (`training/config_v2.yaml`). `training/publish.py` writes `model.sha256` and uploads model, tokenizer, `version.json` and checksums to `s3://models/v1/` and `v2/`; the same files are the assets of GitHub Releases `model-v1` and `model-v2`. The downloaded Release `model.safetensors` matches `models/v1/model.sha256` (`3fdebaf0da34c011…`; `results/release_v1_checksum_verify.txt`) Measured.

Loss per epoch Measured v1: 0.589, 0.368, 0.188, 0.105, 0.080, 0.058, 0.034, 0.022, 0.012, 0.003 | v2: 0.551, 0.342, 0.176, 0.114, 0.090, 0.063, 0.040, 0.027, 0.018, 0.006, 0.002. Training time v1 33 min, v2 32 min on 4 CPU cores.

Model (validation.jsonl, 100 commands)	JSON valid	Intent	Date	Time	Place	Full frame	Note
Teacher phi3.5 (same prompt/settings)	100	83	75	88	89	46	results/teacher_validation.json
Pythia-70M, untrained	0	0	0	0	0	0	prompt + greedy decoding
Student v1, raw output (strict)	98	83	79	89	85	52	student_v1_validation.json
Student v1, as served (strict + salvage)	100	85	81	91	85	52	98 strict, 2 salvaged
Student v2, raw output (strict)	100	80	79	91	85	48	student_v2_validation.json
Student v2, as served	100	80	79	91	85	48	

All numbers Measured. **What changed.** The untrained model continues the text with unrelated prose or a lone "{"; it never produces the format (0% everywhere). After distillation the student reproduces the frame format almost perfectly and copies slot phrases from the command. Its full-frame score (52%) is close to, here even slightly above, the teacher's (46%): it learned the teacher's behaviour, including its labelling conventions, and on 100 examples a few points are within noise. The student cannot be better than the teacher in general; the gap to the gold labels (intent 83%) is inherited from the teacher (83%).

Sequence-level vs. logit-based distillation. (a) *Sequence-level KD* (Kim & Rush, 2016) trains the student with ordinary cross-entropy on the teacher's *generated output sequence*, i.e. the teacher's single best answer is the hard target. (b) *Logit-based (token-level) KD* trains the student to match the teacher's full *probability distribution over the vocabulary at every position* (e.g. KL divergence between softened teacher and student logits), which also transfers how confident the teacher was in the alternatives. (c) It is not possible here because the teacher is used only through Ollama's generate API, which returns text, not per-token distributions; moreover phi3.5 and Pythia use different tokenizers/vocabularies, so their distributions are not position-by-position comparable without an alignment step. (d) A real limitation of this approach: the student learns only the teacher's top-1 answer, including the teacher's mistakes (e.g. the 139 normalised slots that were filtered out show the teacher's systematic errors; the ones that pass the filter are learned as truth), and nothing about uncertainty; a 70M model also falls into repetition loops on a few inputs (2/100 on validation in strict mode).

Task 3: The inference service in a container

Implementation. `service/app.py` (FastAPI, uvicorn, port 8000). Configuration comes only from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS` (+ `FAULT_INJECT`); credentials (`AWS_ACCESS_KEY_ID/SECRET`) come from the Secret `s3-credentials`. At start-up a background thread downloads every object under `MODEL_URI` from SeaweedFS and loads the model; `/healthz` answers 200 at once, `/readyz` answers 503 until the model is loaded. `POST /parse` runs greedy decoding (prompt `Command: ...\nJSON:`), cuts the text at the first newline/closing brace and checks it against `schema.json` (exactly the four keys, intent in the eight, slots null or string). If that strict check fails, a deterministic **salvage** step takes the first allowed intent and the first value of each slot *from the model's own text*, keeping a slot only if it occurs in the command (else null); if no allowed intent exists, the service returns `500 {"error": "invalid_model_output"}`. Valid responses carry `X-Model-Version` and `X-Served-By` (Pod name from the downward API). With `FAULT_INJECT=1` the output is replaced by `not json` before checking and the service returns exactly that 500 body Measured. The image (`python:3.11-slim`, CPU-only PyTorch wheel, every dependency pinned in `service/requirements.txt`, `pip install --no-deps`) contains no weights (0 weight files

found) and is 1.69 GB Measured; it builds from a fresh clone with `make images` or `docker build -f service/Dockerfile .`.

Running under limits (`docker run --cpus=1 --memory=1g --memory-swap=1g`, 50 sequential requests, `results/task3_container.log`) Measured: status codes {200: 50}; median latency 390 ms, max 892 ms; **peak memory 816.5 MiB** (cgroup `memory.peak`, includes page cache of the 282 MB weight download); OOMKilled = false. Before the salvage step was added, the same test gave 49 × 200 and 1 × 500 (`results/before_salvage/task3_container.log`).

Limitations / improvements. Weights are stored in float32 (282 MB); float16/int8 would halve start-up download and memory. Generation is serialised by a lock (one request at a time per pod, by design for a 1-CPU limit). Salvage hides a model weakness; constrained (grammar-guided) decoding would remove it at the source.

Task 4: Kubernetes deployment, scaling and reliability

Manifests (`k8s/app/`): Deployment `kd-service` (2 replicas; requests 500m CPU / 512Mi, limits 1 CPU / 1Gi; readiness `/readyz` every 2 s; liveness `/healthz`; rolling update `maxSurge 1, maxUnavailable 0`; a `preStop` sleep of 8 s so a Pod leaves the Service before it stops), a ClusterIP Service, a Traefik Ingress routing `/parse`, and an HPA (min 1, max 2, CPU 70% of the request; scale-down window 60 s so the test finishes in minutes). All evidence is in `results/task4_all.log` and the files named below; each test was driven by `scripts/task4_k8s.sh`, with the standard Locust settings (4 users) against the Ingress.

- **Readiness gating** Measured: during cold starts the Service had *no* ready endpoints until `/readyz` turned 200 (`ready-endpoints=[]` lines in the log).
- **Cold start** Measured: Pod creation → Ready over three runs (all pods deleted each time): 6, 4, 5 s, **median 5 s**. The logs show the download from object storage, e.g. `Downloading model from object storage: s3://models/v1 (endpoint http://seaweedfs-s3.data.svc.cl` and `Model download complete: 8 files in 1.4s`.
- **HPA** Measured: replica count over the test 1 → 2 → 1; utilisation peaked at 200%/70% of the request and fell to 0% after the load (`hpa_watch.txt`, `hpa_events.txt`).
- **Rolling update** Measured: under load, `MODEL_URI`/`MODEL_VERSION` were changed v1→v2 (headers during: v1, v2, v2, v1, v1, v1), then `kubectl rollout undo`; after rollback the headers show v1, v1, v1.
- **Recovery** Measured: one Pod was deleted under load; the Deployment went back to 2/2 ready (`task4_all.log`).
- **Load balancing** Measured: with exactly two Ready replicas (HPA `minReplicas` temporarily 2) each replica served the share shown below.

Run (2-6 min, 4 users)	Requests	Failed	Error rate	p50 ms	p95 ms	req/s	Model versions	Share per replica
HPA scale test	900	0	0.00%	1300	1700	3.77	v1: 902	55.3%, 44.7%
Load balancing (2 replicas)	497	0	0.00%	930	1400	4.18	v1: 501	49.9%, 50.1%
Rolling update v1→v2→undo	1463	0	0.00%	950	1400	4.07	v1: 1273, v2: 192	5.1%, 6.3%, 6.8%, 6.3%, 38.2%, 37.3%
Recovery (pod deleted)	723	0	0.00%	990	1600	4.04	v1: 725	8.7%, 50.9%, 40.4%
Same tests before the salvage step (results/before_salvage/): error rates hpa_load 1.96%, lb_test 2.00%, rolling 1.75%, recovery 2.03%. These failures were all `500 invalid_model_output` for the two validation commands on which the model loops (Locust cycles through the 100 validation commands, so 2% of requests hit them); the infrastructure itself caused no failures in either set of runs.								

Limitations. This is a **single-node cluster on one laptop**: it demonstrates Pod-level recovery, not fault tolerance across machines (losing the laptop loses everything, including SeaweedFS, which is a single replica on a local-path volume). HPA max 2 × 1 CPU limit and the host's other load bound throughput; metrics-server reports with a ~15-60 s delay, so scale-up lags the load. Assumption Behaviour on a multi-node cloud cluster would differ (image pulls, network latency).

Task 5: Subnetting and network access control

Allocation from 10.20.0.0, largest first, each block right after the previous one, smallest prefix with $2^{32-p}-2 \geq$ hosts (working in `network/subnets.md`):

Subnet	Hosts needed	Network	Prefix	Usable range	Broadcast	Usable hosts
app	1000	10.20.0.0	/22	10.20.0.1 to 10.20.3.254	10.20.3.255	1022

public	250	10.20.4.0	/24	10.20.4.1 to 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 to 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 to 10.20.5.158	10.20.5.159	30

NACL of the public subnet (`network/nacl.json`). Inbound: 50 deny all from 198.51.100.0/24; 100/110 allow TCP 80/443 from 0.0.0.0/0; 120 allow TCP 22 from management 10.20.5.128/27; 130 allow TCP 1024-65535 from app 10.20.0.0/22 (replies from the app's port 8000 to the load balancer's ephemeral ports); everything else falls to the default deny. Outbound: 100 allow TCP 8000 to 10.20.0.0/22; 110 allow TCP 1024-65535 to 0.0.0.0/0 (replies to clients' ephemeral ports). `nacl_sim.py --check` on `packets_sample.jsonl`: **6 / 6 = 100.0%** Measured.

Why replies need their own rules. A NACL is *stateless*: it does not remember that it allowed a request, so the reply packet is checked again on its own, against the rules of its direction. A client's request to port 443 is allowed inbound, but the answer travels outbound to the client's ephemeral port (1024-65535), and the app's answer to the load balancer arrives inbound on the balancer's ephemeral port; without rules 110 (outbound) and 130 (inbound) those replies would hit the default deny. **Why order matters.** Rules are evaluated from the lowest number and the first match wins. A packet from 198.51.100.9 to port 443 matches both "deny 198.51.100.0/24" and "allow 443 from anywhere"; the deny only works because its number (50) is lower than the allow (110). Numbered after the allow, it would never be reached.

Task 6: Reproducibility, testing and CI/CD

Reproducibility. From a fresh clone: `git clone https://github.com/Arunendra21/kd-command-parser.git`, `cd kd-command-parser`, `make up` (3 commands). `make up` creates the cluster, secrets, SeaweedFS, buckets and dataset, builds and imports both images, downloads `model-v1`/`model-v2` from GitHub Releases (verifying `model.sha256`) into `s3://models/`, and deploys the app; `make down` deletes the cluster. A Codespaces `devcontainer.json` provides Docker, k3d, kubectl and gh. **CI** (`.github/workflows/ci.yml`), stages in order: lint (ruff) → unit tests (16 pytest tests: schema validation, salvage, cleaning rules (a)-(e) and count identity, sharding coverage, API headers/FAULT_INJECT/readiness, NACL) → evaluation (downloads the model from the GitHub Release, verifies its checksum, runs `predict.py` + `score.py` on validation) → quality gate (`score.py --min-validity 95`), plus a gitleaks secret scan (reports no leaks) Measured.

The gate doing its job Measured: run with the deliberately bad model (untrained Pythia-70M) fails at the quality gate with `QUALITY GATE FAILED: json_validity 0.0 < 95.0`: https://github.com/Arunendra21/kd-command-parser/actions/runs/42193836249. The same workflow with `model-v1` passes all stages: https://github.com/Arunendra21/kd-command-parser/actions/runs/42193828050. AI use is documented in `AI_USE.md`.

Limitations / improvements. CI does not start a cluster; an end-to-end job (k3d inside the runner, Locust smoke test) would catch manifest regressions. Images are built locally and imported, not pushed to a registry.

Task 7: Performance, cost and review

Standard load test (`locust ... --users 4 --spawn-rate 4 --run-time 2m` against the Ingress; `results/loadtest_stats.csv`, `loadtest_headers.json`, `loadtest_locust.log`) Measured: **p50 960 ms, p95 1400 ms**, throughput **4.07 req/s**, **485 requests, 0 failed (error rate 0.00%)**; share per replica: kd-service-6797f8b74-cg8ns: 49.8%, kd-service-6797f8b74-25mgd: 50.2%. Pod resources during the run (`kubectl top pods` every 10 s, `task7_top.txt`): kd-service-6797f8b74-25mgd: CPU 358-1000m, memory 362-384 MiB; kd-service-6797f8b74-cg8ns: CPU 103-1000m, memory 400-439 MiB. Cold-start median (Task 4): 5 s.

Cost (`data/rate_card.json`: INR 3.50 per vCPU-hour, INR 0.45 per GiB-hour; each replica billed for its limits) Measured for throughput, Assumption that the replicas observed in the run are the replicas running:

```
replicas=2  cpu_limit=1.0 vCPU  mem_limit=1.0 GiB
per replica: 1.0 x 3.5 + 1.0 x 0.45 = 3.9500 INR/h
all replicas: 2 x 3.9500 = 7.9000 INR/h
requests/hour: 4.074 rps x 3600 = 14666.4
cost per 1000 requests = 7.9000 / 14666.4 x 1000 = 0.5386 INR
```

SLO. Target: **99% of /parse requests succeed** (HTTP 200 within 30 s), window: one standard load-test run (2 min). Error budget = 1% × 485 requests = **4.85 requests**; used 0 failed request(s) = 0% of the budget Measured.

Component	Evidence (measured)	Limitation found	Improvement

Data and storage	3000 teacher replies → 2811 kept; 139 rejected by rule (d); checksum reproduced	SeaweedFS is one replica on a node-local volume; teacher reachable only through a tunnel	Replicated SeaweedFS (3 volume servers) or managed S3; teacher inside the cluster
Model quality	Student v1 full frame 52% vs teacher 46%; strict JSON validity 98%	Inherits teacher errors; repetition loops on ~1-2% of inputs	Grammar-constrained decoding; more/cleaner data; logit-level KD with a same-tokenizer teacher
Latency and throughput	p50 960 ms, p95 1400 ms, 4.07 req/s with 4 users	One request at a time per pod; max 2 replicas	int8/ONNX inference, KV-cache reuse, higher max replicas, batching
Resource utilisation	max 1000m CPU / 384 MiB; max 1000m CPU / 439 MiB per pod; Task 3 peak 816.5 MiB	CPU-bound at the 1-CPU limit while memory is half used	Lower memory limit (cheaper), float16 weights, right-size requests
Reliability and recovery	Cold start median 5 s; recovery error rate 0.00%; rolling update error rate 0.00%	Single node: no protection against host failure	Multi-node cluster, PodDisruptionBudget, anti-affinity, readiness on a warm-up request
Security and cost	gitleaks: no leaks; cost INR 0.5386 per 1,000 requests	Plain-HTTP S3 inside the cluster; one admin S3 identity; replicas billed while idle	TLS, per-service least-privilege S3 keys, scale to zero / lower limits when idle

References

1. Y. Kim and A. M. Rush. Sequence-Level Knowledge Distillation. EMNLP 2016.
2. Y.-C. Chen et al. Distilling Knowledge Learned in BERT for Text Generation. ACL 2020.
3. G. Hinton, O. Vinyals, J. Dean. Distilling the Knowledge in a Neural Network. 2015.
4. E. Bastianelli et al. SLURP: A Spoken Language Understanding Resource Package. EMNLP 2020.
5. S. Biderman et al. Pythia: A Suite for Analyzing Large Language Models. ICML 2023; EleutherAI/pythia-70m model card.
6. Microsoft. Phi-3 Technical Report, 2024; Ollama library phi3.5.
7. PyTorch Knowledge Distillation Tutorial; Hugging Face Transformers causal LM guide; TRL Distillation Trainer docs.
8. k3d, K3s, SeaweedFS, Kubernetes (Deployments, Services, Probes, Secrets, HPA, resource metrics), Traefik, Locust, Gitleaks documentation.
9. AWS VPC User Guide: Network ACLs. Google SRE Book: Service Level Objectives.

Appendix A: Commands

```
make up | make label OLLAMA_URL=... SHARD_COUNT=3 | make clean-data | make teacher-eval OLLAMA_URL=...
make train SUBSET=50 | make train | make train-v2 | make publish VERSION=v1 | make release VERSION=v1
make eval-base | make eval-student VERSION=v1 | make task3 | make task4 | make task7 | make test | make
down
```

Appendix B: Teacher prompt (prompts/teacher_prompt.txt, first lines)

```
You are the command parser of a voice assistant. Read one short English command and reply with exactly one
JSON object that has exactly these four keys, in this order: "intent", "date", "time", "place".

"intent" must be exactly one of these eight values:
- "alarm_set": create an alarm (set my alarm for six am tomorrow)
- "alarm_query": ask about existing alarms (what alarms do i have set for today)
- "calendar_set": add an event or a reminder (add running to my calendar for thursday)
- "calendar_query": ask about scheduled events (what are my plans for the month of may)
- "calendar_remove": d…
```